COURSE NAME

OBJECT ORIENTED ANALYSIS AND DESIGN

CSC 2210

(UNDERGRADUATE)

UNIFIED MODELING LANGUAGE

CHAPTER 2

INTRODUCTION TO UML

MOHAIMEN-BIN-NOOR
ASSISTANT PROFESSOR, CS
mohaimen.niloy@aiub.edu

UML

- Unified Modeling Language
- UML is a standardized general-purpose modeling language in the field of **software engineering**. The standard is managed, and was created by, the Object Management Group (OMG) (http://www.omg.org/)
- UML website: http://www.uml.org
- Increase understanding/communication of product to customers and developers
- Support for UML in many software packages today (e.g., UMLet, Rational Rose, ArgoUML, etc.)

WHY USE UML

The UML is a language for Visualizing

- Some things are best modeled textually; others graphically.
- One developer can write a model in the UML, and another developer, or even another tool, can interpret that model unambiguously

The UML is a language for Specifying

- Specifying means building models that are precise, unambiguous, and complete
- In particular, the UML addresses the specification of all the important analysis, design, and implementation decisions that must be made in developing and deploying a software-intensive system.

WHY USE UML

The UML is a language for Constructing

- UML is not a visual programming language, but its models can be directly connected to a variety of programming languages
- It is possible to map from a model in the UML to a programming language such as Java, C++, or VB, or even to tables in a RDBMS

The UML is a language for Documenting

- The UML addresses the documentation of a system's architecture and all of its details
- The UML also provides a language for expressing requirements and for tests
- Finally, the UML provides a language for modeling the activities of project planning and release management

BUILDING BLOCKS OF UML

- The vocabulary of the UML encompasses three kinds of building blocks:
  1. **Things** - abstractions that are the primary concern in a model
     - Structural things - nouns/static parts of UML models
     - Behavioral things - verbs/dynamic parts of UML models
     - Grouping things - organizational parts of UML models
     - Annotation things - explanatory parts of UML models
  2. **Relationships** - tie the things together
  3. **Diagrams** - group collections of things

STRUCTURAL THINGS

- **Structural things** are the nouns of the UML model. These are the mostly static parts of a model, representing elements that are either conceptual or physical. There are **Eight** kinds of structural things.

1. A Class

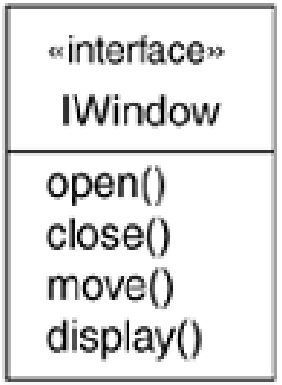
- A class can be defined as a template/blueprint that describes the behaviors/states that object of its type support.
- Description of set of objects that share the same attributes, operations, relationships and semantics

Student
Name ID CGPA DOB
Study ( ) Play ( ) Eating ( )

STRUCTURAL THINGS

2. An interface

- Is a collection of operations that specify a service of a class or component. An interface therefore describes the **externally** visible behavior of that element.
- You cannot instantiate an interface. Therefore, interface does not contain any constructors.
- An interface is not extended by a class; it is implemented by a class (unchanged in implementation).
- An interface can extend multiple interfaces.



STRUCTURAL THINGS

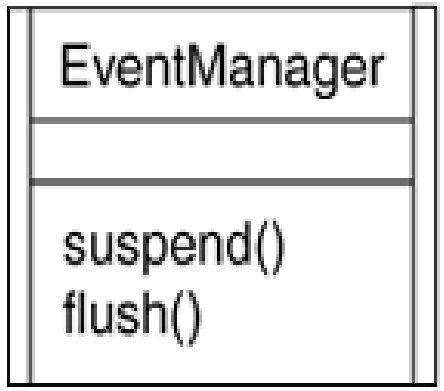
3. **A collaboration:** A collection of UML building blocks (classes, interfaces, relationships) that work together to provide some functionality within the system. (e.g., Interoperability, block chain technology to conned external system in providing one-stop e-services

4. **A Use case** is a description of set of sequence of actions that a system performs that yields an observable result of value to a particular **actor**.

STRUCTURAL THINGS

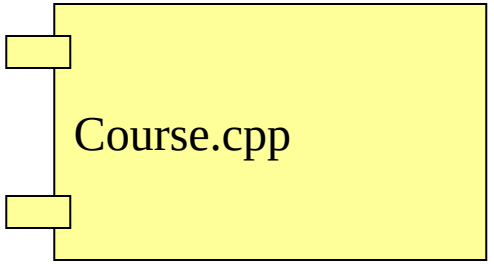
5. An active class

- is a class whose objects (known active object) own one or more processes or threads and therefore can initiate control activity (*modify variables, change program behavior, and so on*).
- Active object initiate and control the flow of activity, while passive object wait for another object to call them generally for store data and serve other classes.
- Graphically, an active class is rendered as a class with double lines on the left and right



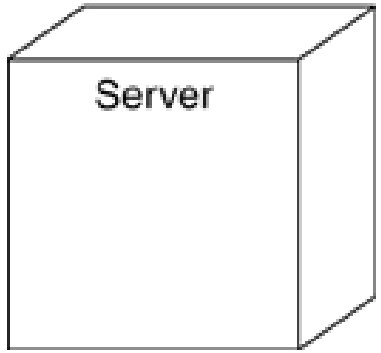
STRUCTURAL THINGS

6. A Component is a packaging of classes, interfaces, and collaboration. Total program is divided into various parts or module.



7. An artifact is a physical and replaceable part of a system that contains physical information ("bits").

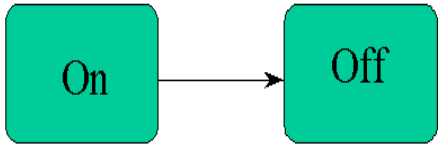
8. A Node: Computational resources that exists at run time, typically h/w resources e.g., memory, processor, etc.



BEHAVIORAL THINGS

- **Behavioral things** are the verbs of the UML model. These are the mostly dynamic parts of a model, representing behavior over time and space. There are **three** kinds of behavioral things.

  - **A state machine** is a behavior that specifies the sequences of states an object or an interaction goes through during its lifetime in response to events.

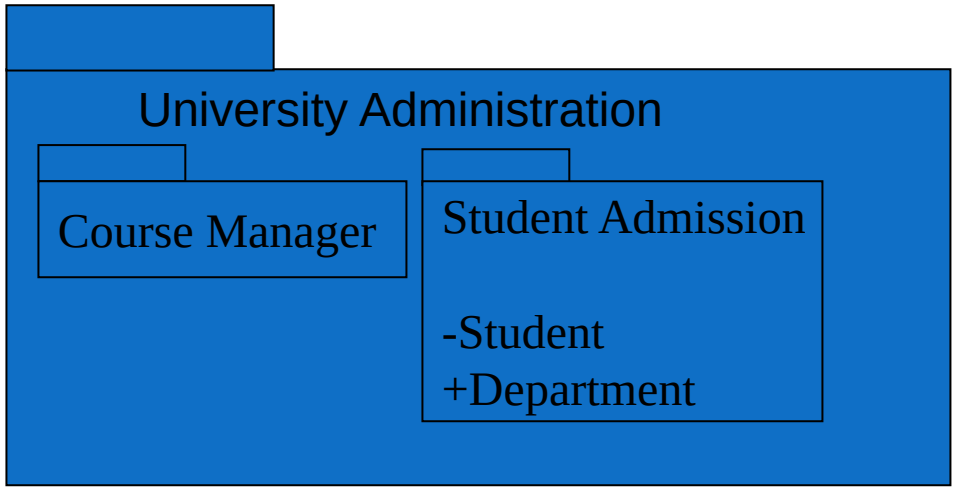


  - **An Interaction** is a set of messages exchanged among a set of objects to accomplish a specific purpose

  - An **activity** is a behavior that specifies the sequence of steps a computational process performs.

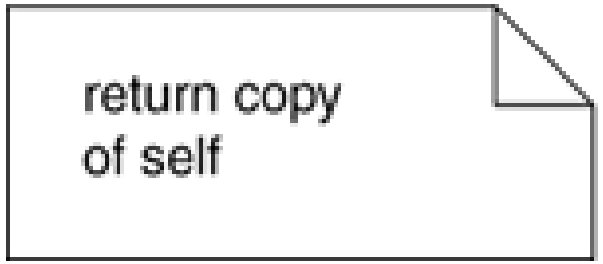
GROUPING THINGS

- The organizational parts of UML models. These are the boxes into which a model can be decomposed. There is one primary kind of grouping thing, namely, **packages**.
- Organizing elements (structural/behavioral) into groups.
- Purely conceptual. only exists at development time. And can be nested.
- Variations of packages are: Frameworks, models, & subsystems.



ANNOTATIONAL THINGS

- Annotational things are the **explanatory** parts of UML models.
- These are the comments you may apply to describe, illuminate, and remark about any element in a model.
- There is one primary kind of Annotational thing, called a **note**.



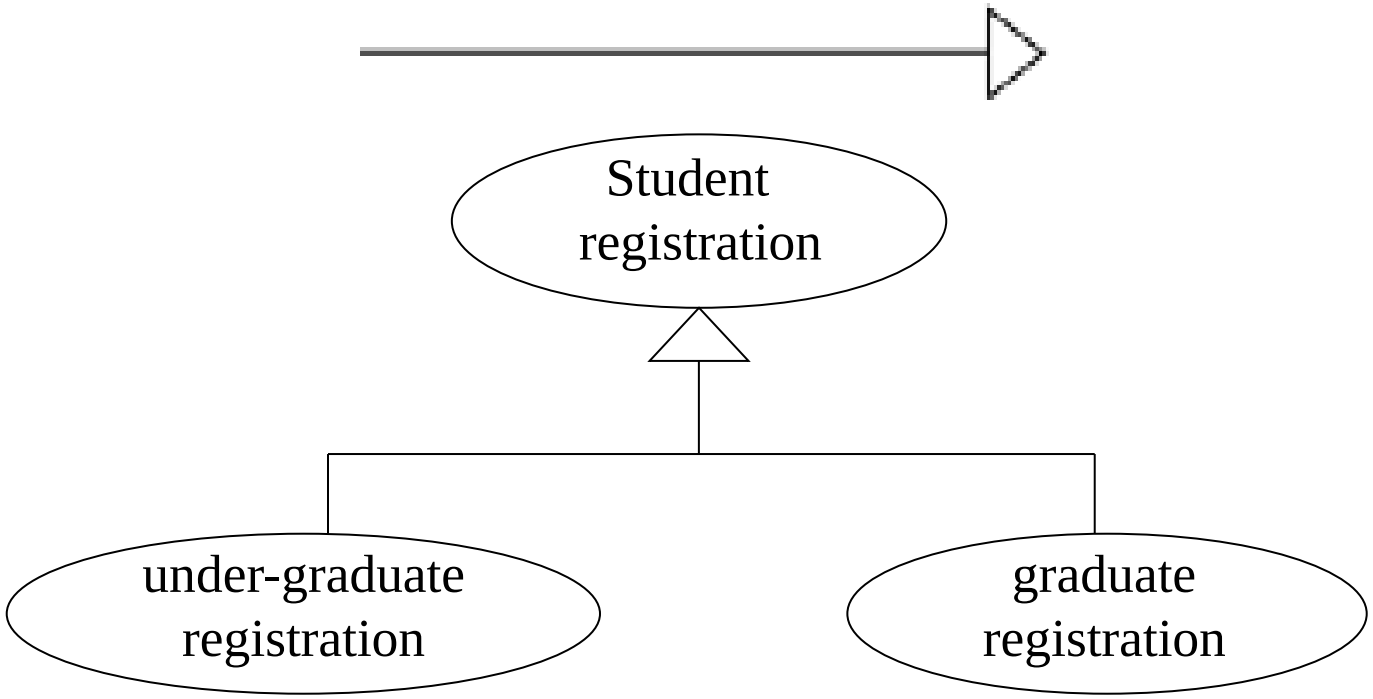
RELATIONSHIPS IN UML

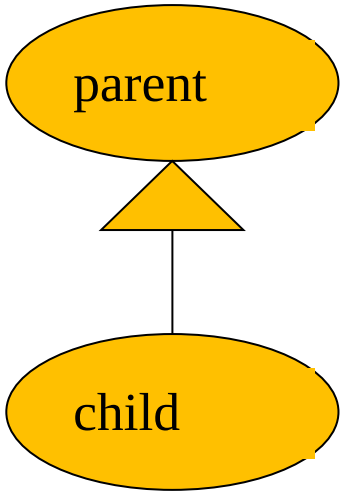
- There are four kinds of relationships in the UML
  - Association
  - Generalization
  - Realization
  - Dependency

Association: is a structural relationship that describes a set of links. A link is a connection between objects.

RELATIONSHIPS IN UML

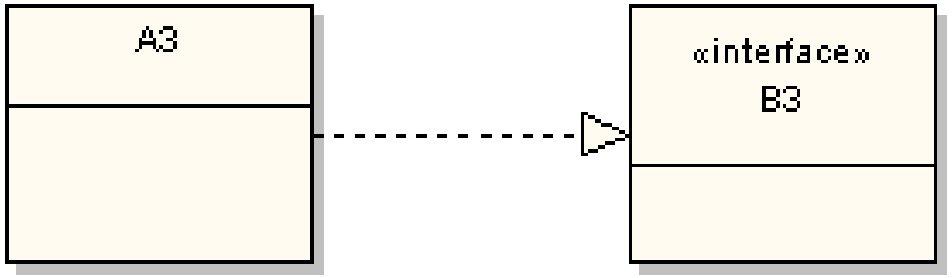
Generalization: can be defined as a relationship which connects a specialized element with a generalized element. It basically describes inheritance (IS-A) relationship in the world of objects.





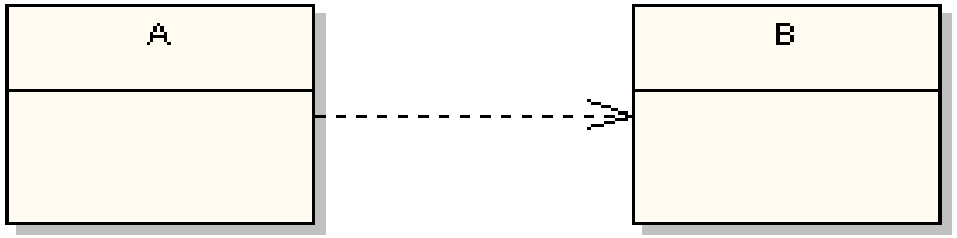
RELATIONSHIPS IN UML

Realization: denotes the implementation (interface) of the functionality defined in one class by another class.




```
class A3 implements B3
```

Dependency: is a relationship between two things in which change in one element also affects the other one.




```
class A depends on class B
```

DIAGRAMS IN UML

A **diagram** is the graphical presentation of a set of elements, most often rendered as a connected graph of vertices (things) and arcs (relationships).

1. Class Diagram – Structural. Represents Static Design View or Static Process View.
2. Object Diagram – Structural. Represents Static Design View or Static Process View.
3. Component Diagram – Structural. Represents Static Implementation View.
4. Artifact Diagram – Structural. Represents Static Implementation View.
5. Use Case Diagram – Behavioral. Represents Static Use Case View.
6. Deployment Diagram – Structural. Represents Static Deployment View.

DIAGRAMS IN UML

A **diagram** is the graphical presentation of a set of elements, most often rendered as a connected graph of vertices (things) and arcs (relationships).

7. Interaction Diagram – Behavioral. Represents Dynamic View.
8. Sequence Diagram – Interaction Diagram emphasizing on time-ordering of messages.
9. Communication Diagram – Interaction Diagram emphasizing on structural organization of objects.
10. State Diagram – Behavioral. Represents Dynamic View of an object.
11. Activity Diagram – Behavioral. Represents Dynamic View.

DIAGRAMS IN UML

A **diagram** is the graphical presentation of a set of elements, most often rendered as a connected graph of vertices (things) and arcs (relationships).

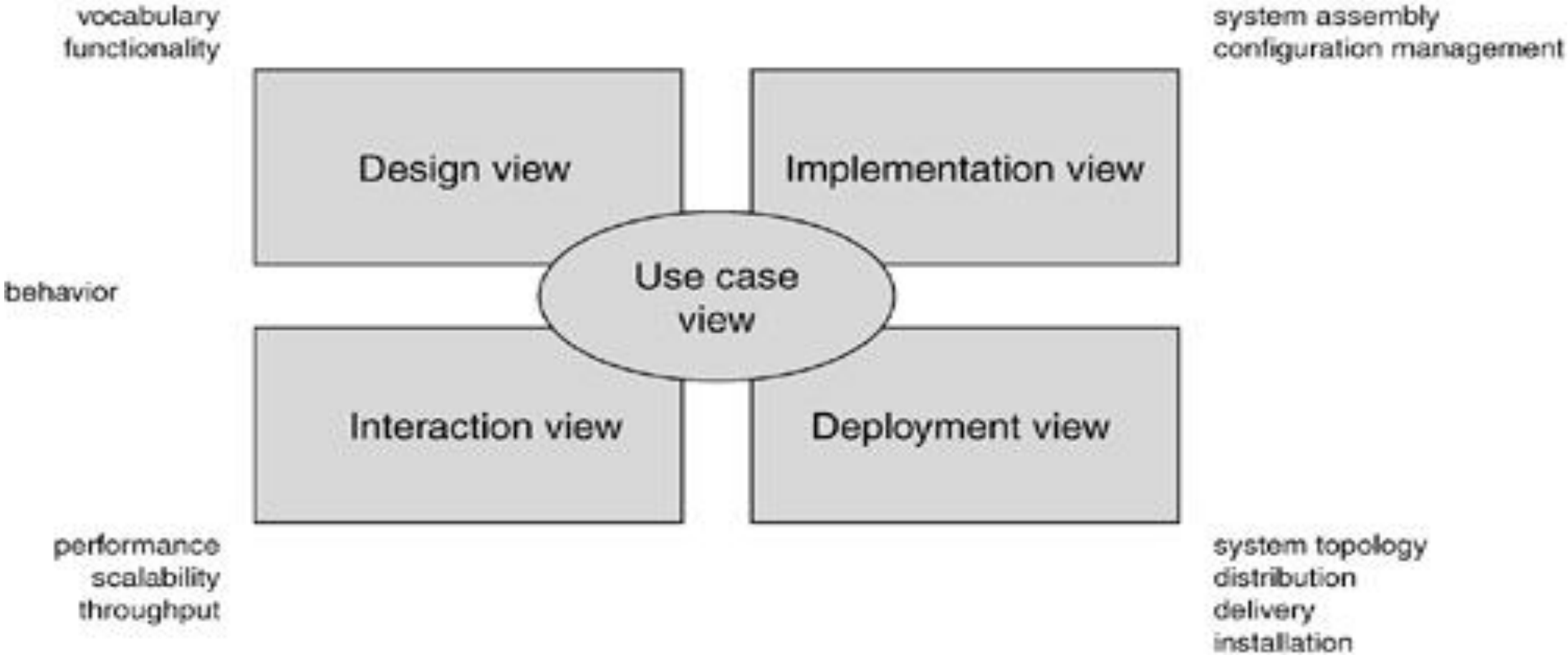
12. Package Diagram – Represents Static View. Shows decomposition of model.
13. Timing Diagram – Interaction Diagram with specific time.
14. Interaction Overview Diagram – Hybrid of Interaction and Activity Diagram.
15. Composite structure diagram – Hybrid Diagram.

ARCHITECTURE OF OO SOFTWARE SYSTEM

- To understand the architecture of object oriented software system, one needs several complementary and interlocking views.

- Each of these views may have structural, as well as behavioural, aspects. Together these views represent the blueprints of Object Oriented Software System.

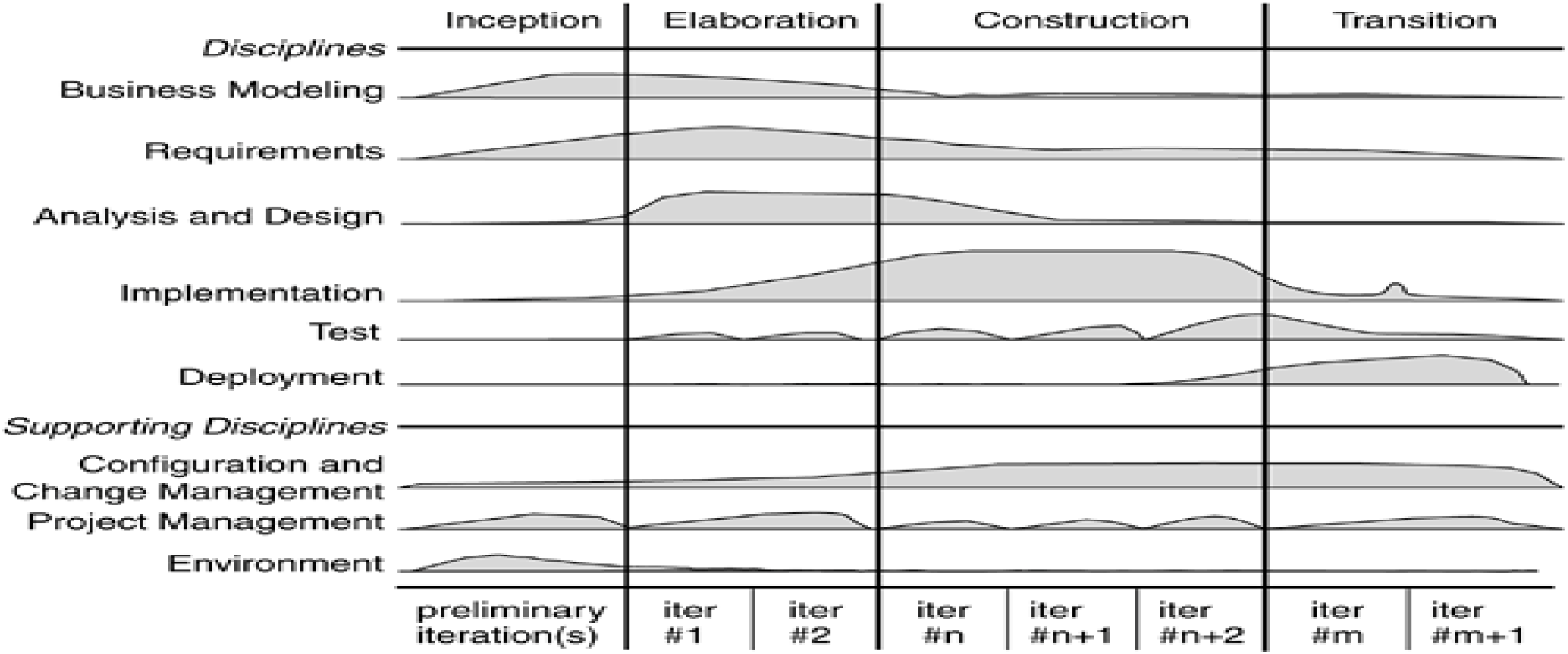
SYSTEM ARCHITECTURE MODELING



ARCHITECTURE OF OO SOFTWARE SYSTEM

- A use case view – exposes the behaviour of the system.
- A design view – captures the vocabulary and functionality of the system.
- A process view – models the distribution of the system's processes and threads and addresses the performance and scalability.
- An implementation view – addresses the physical realization of the system, system assembly and configuration management.
- A deployment view – focuses on system topology, distribution, delivery and installation.

SDLC – RATIONAL UNIFIED MODEL (RUP)



SDLC – RATIONAL UNIFIED MODEL (RUP)

- **Inception** is the first phase of the process, when the seed idea for the development is brought up to the point of being at least internally sufficiently well-founded to warrant entering into the elaboration phase.
- **Elaboration** is the second phase of the process, when the product requirements and architecture are defined. In this phase, the requirements are articulated, prioritized, and baselined. A system's requirements may range from general vision statements to precise evaluation criteria, each specifying particular functional or nonfunctional behavior and each providing a basis for testing.
- **Construction** is the third phase of the process, when the software is brought from an executable architectural baseline to being ready to be transitioned to the user community. Here also, the system's requirements and especially its evaluation criteria are constantly reexamined against the business needs of the project, and resources are allocated as appropriate to actively attack risks to the project.

SDLC – RATIONAL UNIFIED PROCESS (RUP)

- **Transition** is the fourth phase of the process, when the software is delivered to the user community. Rarely does the software development process end here, for even during this phase, the system is continuously improved, bugs are eliminated, and features that didn't make an earlier release are added.
- **Iteration** is a distinct set of work tasks, with a baselined plan and evaluation criteria that results in an executable system that can be run, tested, and evaluated.

REFERENCES

- Booch, G., Rumbaugh, J. & Jacobson, I. (2005). *The unified modeling language user guide*. Pearson Education India.